

HDOC – Experiment 18

Question

Write a menu-driven C program using switch-case to perform one of these operations:

- 1. Check whether a number is a Strong number**
- 2. Check whether a number is an Armstrong number**
- 3. Check whether a number is a Harshad number**
- 4. Check whether a number is a Duck number**

❖ Problem understanding

Each operation must have:

- A separate user-defined function
- void return type
- No arguments/parameters
- The function itself reads the number, processes it, and displays the result

❖ Algorithm

Main Program Algorithm

1. Start.
2. Declare integer variable choice.

3. Display the menu:

-

1. Strong Number

-

2. Armstrong Number

-

3. Harshad Number

-

4. Duck Number

4. Accept choice from the user.

5. Use switch(choice).

6. If choice is 1, call strongNumber().

7. If choice is 2, call armstrongNumber().

8. If choice is 3, call harshadNumber().

9. If choice is 4, call duckNumber().

10. For any other choice, display "Invalid choice".

11. Stop.

❖ **Algorithm for strongNumber()**

1. Start.

2. Input a positive integer num.

3. Store num in original.

4. Initialize sum = 0.

5. Repeat while num > 0:

- Find the last digit using digit = num % 10.

- Initialize fact = 1.

- Calculate factorial of digit.
 - Add factorial to sum.
 - Remove the last digit using $\text{num} = \text{num} / 10$.
6. Compare sum with original.
 7. If they are equal, display Strong number.
 8. Otherwise, display Not a Strong number.
 9. Stop.

❖ **Algorithm for armstrongNumber()**

1. Start.
2. Input a positive integer num.
3. Store num in original.
4. Count the number of digits and store it in count.
5. Restore the number using $\text{num} = \text{original}$.
6. Initialize $\text{sum} = 0$.
7. Repeat while $\text{num} > 0$:
 - Extract the last digit.
 - Initialize $\text{power} = 1$.
 - Multiply power by the digit count times.
 - Add power to sum.
 - Remove the last digit.
8. Compare sum with original.
9. If equal, display Armstrong number.
10. Otherwise, display Not an Armstrong number.
11. Stop.

❖ **Algorithm for harshadNumber()**

1. Start.
2. Input a positive integer num.
3. Store num in original.
4. Initialize sum = 0.
5. Repeat while num > 0:
 - Extract the last digit using num % 10.
 - Add the digit to sum.
 - Remove the last digit using num / 10.
6. Check whether original % sum == 0.
7. If true, display Harshad number.
8. Otherwise, display Not a Harshad number.
9. Stop.

❖ **Algorithm for duckNumber()**

1. Start.
2. Input a positive integer num.
3. Initialize found = 0.
4. Repeat while num > 0:
 - Extract the last digit using num % 10.
 - If the digit is 0, set found = 1 and terminate the loop.
 - Otherwise, remove the last digit.
5. If found == 1, display Duck number.
6. Otherwise, display Not a Duck number.
7. Stop.

❖ Test case table

Test Case	Choice	Input	Expected Output	Actual Output	Result
1	1	145	Strong number	SAME	PASS
2	1	123	Not a strong number	SAME	PASS
3	2	153	Armstrong number	SAME	PASS
4	2	123	Not an Armstrong number	SAME	PASS
5	3	18	Harshad number	SAME	PASS
6	3	19	Not a Harshad number	SAME	PASS
7	4	1025	Duck number	SAME	PASS
8	4	1235	Not a duck number	SAME	PASS

❖ Program code in C

```
#include <stdio.h>
```

```
/* Function declarations */
```

```
void strongNumber();
```

```
void armstrongNumber();
```

```
void harshadNumber();
```

```
void duckNumber();
```

```
int main()
{
    int choice;

    printf("DIGIT BASED OPERATIONS\n");
    printf("1. Check Strong Number\n");
    printf("2. Check Armstrong Number\n");
    printf("3. Check Harshad Number\n");
    printf("4. Check Duck Number\n");

    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            strongNumber();
            break;

        case 2:
            armstrongNumber();
            break;

        case 3:
            harshadNumber();
            break;

        case 4:
```

```
        duckNumber();

        break;

    default:

        printf("Invalid choice\n");
    }

    return 0;
}


/* Function to check Strong Number */
void strongNumber()
{
    int num, original, digit;
    int sum = 0, fact, i;

    printf("Enter a positive integer: ");
    scanf("%d", &num);

    original = num;

    while (num > 0)
    {
        digit = num % 10;

        fact = 1;
```

```
    for (i = 1; i <= digit; i++)
    {
        fact = fact * i;
    }

    sum = sum + fact;

    num = num / 10;
}

if (sum == original)
    printf("%d is a Strong number.\n", original);
else
    printf("%d is not a Strong number.\n", original);
}
```

```
/* Function to check Armstrong Number */
```

```
void armstrongNumber()
```

```
{
```

```
    int num, original, temp, digit;
```

```
    int count = 0, sum = 0;
```

```
    int power, i;
```

```
    printf("Enter a positive integer: ");
```

```
    scanf("%d", &num);
```

```
    original = num;
```

```
    temp = num;
```



```
/* Count number of digits */
while (temp > 0)
{
    count++;
    temp = temp / 10;
}

temp = num;

while (temp > 0)
{
    digit = temp % 10;

    power = 1;

    for (i = 1; i <= count; i++)
    {
        power = power * digit;
    }

    sum = sum + power;
    temp = temp / 10;
}

if (sum == original)
    printf("%d is an Armstrong number.\n", original);
else
```

```
    printf("%d is not an Armstrong number.\n", original);  
}
```

```
/* Function to check Harshad Number */
```

```
void harshadNumber()
```

```
{
```

```
    int num, original, digit;
```

```
    int sum = 0;
```

```
    printf("Enter a positive integer: ");
```

```
    scanf("%d", &num);
```

```
    original = num;
```

```
    while (num > 0)
```

```
{
```

```
    digit = num % 10;
```

```
    sum = sum + digit;
```

```
    num = num / 10;
```

```
}
```

```
    if (original % sum == 0)
```

```
        printf("%d is a Harshad number.\n", original);
```

```
    else
```

```
        printf("%d is not a Harshad number.\n", original);
```

```
}
```

```
/* Function to check Duck Number */  
void duckNumber()  
{  
    int num, original, digit;  
    int found = 0;  
  
    printf("Enter a positive integer: ");  
    scanf("%d", &num);  
  
    original = num;  
  
    while (num > 0)  
    {  
        digit = num % 10;  
  
        if (digit == 0)  
        {  
            found = 1;  
            break;  
        }  
  
        num = num / 10;  
    }  
  
    if (found == 1)  
        printf("%d is a Duck number.\n", original);  
    else
```

```
printf("%d is not a Duck number.\n", original);  
}
```

❖ Compile, Execute and Evaluate

Test case 1

```
(kali㉿kali)-[~/Desktop/exp18]  
$ ./check  
DIGIT BASED OPERATIONS  
1. Check Strong Number  
2. Check Armstrong Number  
3. Check Harshad Number  
4. Check Duck Number  
Enter your choice: 1  
Enter a positive integer: 145  
145 is a Strong number.
```

Test case 2

```
(kali㉿kali)-[~/Desktop/exp18]  
$ ./check  
DIGIT BASED OPERATIONS  
1. Check Strong Number  
2. Check Armstrong Number  
3. Check Harshad Number  
4. Check Duck Number  
Enter your choice: 1  
Enter a positive integer: 123  
123 is not a Strong number.
```

Test case 3

```
(kali㉿kali)-[~/Desktop/exp18]  
$ ./check  
DIGIT BASED OPERATIONS  
1. Check Strong Number  
2. Check Armstrong Number  
3. Check Harshad Number  
4. Check Duck Number  
Enter your choice: 2  
Enter a positive integer: 153  
153 is an Armstrong number.
```

Test case 4

```
(kali㉿kali)-[~/Desktop/exp18]  
$ ./check  
DIGIT BASED OPERATIONS  
1. Check Strong Number  
2. Check Armstrong Number  
3. Check Harshad Number  
4. Check Duck Number  
Enter your choice: 2  
Enter a positive integer: 123  
123 is not an Armstrong number.
```

Test case 5

```
(kali㉿kali)-[~/Desktop/exp18]  
$ ./check  
DIGIT BASED OPERATIONS  
1. Check Strong Number  
2. Check Armstrong Number  
3. Check Harshad Number  
4. Check Duck Number  
Enter your choice: 3  
Enter a positive integer: 18  
18 is a Harshad number.
```

Test case 6

```
(kali㉿kali)-[~/Desktop/exp18]  
$ ./check  
DIGIT BASED OPERATIONS  
1. Check Strong Number  
2. Check Armstrong Number  
3. Check Harshad Number  
4. Check Duck Number  
Enter your choice: 3  
Enter a positive integer: 19  
19 is not a Harshad number.
```

Test case 7

```
(kali㉿kali)-[~/Desktop/exp18]  
$ ./check  
DIGIT BASED OPERATIONS  
1. Check Strong Number  
2. Check Armstrong Number  
3. Check Harshad Number  
4. Check Duck Number  
Enter your choice: 4  
Enter a positive integer: 1025  
1025 is a Duck number.
```

Test case 8

```
(kali㉿kali)-[~/Desktop/exp18]  
$ ./check  
DIGIT BASED OPERATIONS  
1. Check Strong Number  
2. Check Armstrong Number  
3. Check Harshad Number  
4. Check Duck Number  
Enter your choice: 4  
Enter a positive integer: 1235  
1235 is not a Duck number.
```

❖ Compilation and Execution screenshot

```
(kali㉿kali)-[~/Desktop/exp18]
$ vi check.c

(kali㉿kali)-[~/Desktop/exp18]
$ gcc check.c -o check

(kali㉿kali)-[~/Desktop/exp18]
$ gcc -Wall -Wextra check.c -o check

(kali㉿kali)-[~/Desktop/exp18]
$ ./check
DIGIT BASED OPERATIONS
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 1
Enter a positive integer: 145
145 is a Strong number.

(kali㉿kali)-[~/Desktop/exp18]
$ ./check
DIGIT BASED OPERATIONS
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
Enter your choice: 1
Enter a positive integer: 123
123 is not a Strong number.
```

```
(kali㉿kali)-[~/Desktop/exp18]
```

```
$ ./check
```

DIGIT BASED OPERATIONS

1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number

Enter your choice: 2

Enter a positive integer: 153

153 is an Armstrong number.

```
(kali㉿kali)-[~/Desktop/exp18]
```

```
$ ./check
```

DIGIT BASED OPERATIONS

1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number

Enter your choice: 2

Enter a positive integer: 123

123 is not an Armstrong number.

```
(kali㉿kali)-[~/Desktop/exp18]
```

```
$ ./check
```

DIGIT BASED OPERATIONS

1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number

Enter your choice: 3

Enter a positive integer: 18

18 is a Harshad number.


```
(kali㉿kali)-[~/Desktop/exp18]  
$ ./check
```

DIGIT BASED OPERATIONS

1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number

Enter your choice: 3

Enter a positive integer: 19

19 is not a Harshad number.

```
(kali㉿kali)-[~/Desktop/exp18]  
$ ./check
```

DIGIT BASED OPERATIONS

1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number

Enter your choice: 4

Enter a positive integer: 1025

1025 is a Duck number.

```
(kali㉿kali)-[~/Desktop/exp18]  
$ ./check
```

DIGIT BASED OPERATIONS

1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number

Enter your choice: 4

Enter a positive integer: 1235

1235 is not a Duck number.